MANCHESTER 1824
The University of Manchester

A Modular Real-Time Haptic Rendering System Integrating Physics-Based Simulation and OpenGL Visualisation

A dissertation submitted to The University of Manchester for the degree of

Bachelor of Engineering in Mechatronic Engineering

in the Faculty of Science and Engineering

Year of submission

2026

Student ID

11349510

School of Engineering

Abstract

This is abstract text.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Declaration of originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright statement

i The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library's regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University's policy on Presentation of Theses.

Acknowledgments

Acknowledgments go here.

1 Introduction

1.1 Background and Motivation

Robotic manipulators are increasingly used in domains such as industrial automation, teleoperation, and robot-assisted surgery, where safe and effective interaction with physical environments is essential. In many of these contexts, operator training plays a critical role in determining task quality, safety, and efficiency. Although visual feedback provides important global information about the workspace, it is often insufficient for developing fine motor control, accurate force regulation, and intuitive understanding of contact conditions.

Haptic interfaces address this limitation by enabling bidirectional physical interaction between a user and a virtual or remote environment. By rendering forces back to the operator, such systems can communicate contact, constraint, and interaction information that is difficult to convey visually alone [1]. In robotic manipulation tasks, this is particularly valuable for training applications, where users must learn not only where to move, but also how to regulate force and respond to contact conditions. Prior work has shown that haptic feedback can improve task performance and reduce excessive applied force, particularly for less experienced users [2].

Despite these benefits, access to haptic systems remains limited. Commercial haptic interfaces provide mature and high-performance force feedback, but they are typically expensive and frequently tied to proprietary hardware and software ecosystems. This can restrict accessibility, reduce architectural transparency, and make it difficult for researchers and students to experiment with alternative control strategies, custom simulation environments, or modified hardware configurations. As a result, there is strong practical motivation for low-cost and modular haptic platforms that are suitable for prototyping, experimentation, and robotics training research.

At the same time, achieving stable and convincing haptic interaction is technically challenging. Real-time haptic rendering commonly requires update rates on the order of 1 kHz together with low latency and carefully designed control structure in order to render contact stably and responsively [3]. The stability of haptic interaction is particularly sensitive to delay in the feedback loop, since communication and computation delays introduce phase lag and reduce the maximum achievable virtual stiffness [4]. Consequently, the design of a practical haptic system is not simply a matter of adding actuators to an input device: it requires coordinated consideration of mechanical design, embedded control, communication architecture, simulation coupling, and force rendering strategy.

1.2 Problem Definition

This dissertation addresses the problem of how to realise a *low-cost*, *modular*, *real-time haptic interface system* suitable for robotic manipulation training and experimentation, while explicitly accounting for the practical constraints that arise in implementation.

The specific challenge is not merely to construct a haptic input device, but to integrate multiple subsystems into a stable bidirectional interaction loop: user motion must be sensed at the device, transmitted to a host system, interpreted within a simulated environment, converted into interaction forces, mapped back into actuator commands, and rendered to the user in real time. In practice, each stage of this loop introduces constraints. Limited actuator capability affects the achievable force output, communication delay limits responsiveness and stable virtual stiffness, and software architecture determines whether the system remains extensible and maintainable.

Existing work demonstrates the value of haptic feedback and the feasibility of low-cost haptic hardware, but there remains a practical gap between simple hardware demonstrations and fully integrated systems that combine low-cost actuation, real-time communication, physics-based simulation, and explicit treatment of latency and stability. Educational and open-source devices have shown that affordable haptic hardware is possible [5], [6], and low-cost 2-DOF academic systems have also been demonstrated **inproceedings**. However, comparatively less emphasis is placed on complete end-to-end architectures in which hardware, embedded control, host-side simulation, rendering, and communication are developed together and evaluated as an integrated real-time system.

The problem addressed in this project is therefore defined as follows: *to design*, *implement*, *and evaluate a low-cost haptic rendering system that is sufficiently modular for experimentation*, *sufficiently responsive for real-time interaction*, *and sufficiently stable to demonstrate practical bidirectional haptic coupling with a simulated robotic environment*.

1.3 Aim and Scope of This Dissertation

The aim of this project is to design, implement, and evaluate a modular low-cost haptic rendering system integrating a two-degree-of-freedom input device with a custom real-time simulation and visualisation framework.

The project focuses on demonstrating the feasibility of an end-to-end architecture rather than on producing a commercial-grade haptic product. More specifically, the dissertation is concerned with:

- the design and construction of a low-cost actuated 2-DOF haptic device,
- the development of embedded firmware for sensing, communication, and actuator control,
- the implementation of a host-side software framework supporting simulation, haptic rendering, and visualisation,
- the integration of these components into a real-time bidirectional interaction loop, and
- the experimental evaluation of communication performance, timing behaviour, stability, and qualitative interaction characteristics.

The scope of the work is intentionally limited. The dissertation does not attempt to demonstrate fully calibrated force output, high-fidelity commercial-level rendering, or a formal user study of training effectiveness. In addition, while the device was designed as a 2-DOF haptic interface, final hardware validation was performed with force feedback available on only one joint due to actuator failure during development. Accordingly, the main contribution of the work lies in the *system-level design, integration, and evaluation of a modular real-time prototype*, together with analysis of the practical factors that limit its performance.

This framing is important because the value of the dissertation lies not only in demonstrating that low-cost haptic interaction is possible, but also in identifying the architectural and hardware constraints that dominate performance in practice. As the results later show, the principal limitation of the implemented system is not host-side computational speed, but communication latency and the resulting constraint it imposes on achievable haptic stiffness and responsiveness.

1.4 Objectives

To achieve this aim, the project pursued the following objectives:

- to design and construct a 2-DOF haptic device using low-cost brushless actuators, magnetic encoders, and lightweight fabricated structural components;
- to implement embedded firmware for encoder acquisition, actuator control, and bidirectional packet-based communication;
- to develop a modular host-side software architecture supporting world management, physics simulation, haptic rendering, and graphical visualisation;

- to implement proxy-based contact rendering and Jacobian-based force-to-torque mapping for haptic feedback generation;
- to evaluate the resulting system in terms of communication behaviour, closed-loop timing, stability, safety, and qualitative force-feedback performance; and
- to assess the practical limitations of the architecture and identify directions for future improvement.

1.5 Contributions of This Work

The work presented in this dissertation makes the following principal contributions:

- the design and implementation of a low-cost actuated haptic interface prototype based on a 2-DOF mechanical architecture;
- the development of a custom modular host-side software framework integrating haptic rendering, rigid-body simulation, and OpenGL-based visualisation;
- the implementation of a bidirectional real-time interaction pipeline linking the physical device to the simulated environment;
- an experimental evaluation showing that the system can achieve stable real-time interaction at high update rates, while also revealing communication latency as the dominant limitation on performance; and
- a critical analysis of the trade-offs between cost, modularity, stability, and achievable haptic fidelity in a student-scale prototype system.

2 Literature review

2.1 Haptic Interfaces for Robotics

Haptic interfaces play a central role in teleoperation, robot-assisted surgery, training, and interactive virtual environments, where visual feedback alone is often insufficient for the development of accurate motor skills, force regulation, and intuitive understanding of contact conditions [1].
A large body of empirical evidence demonstrates that the inclusion of haptic feedback improves task performance across a wide range of robotic manipulation tasks. For example, a recent meta-analysis of robot-assisted surgery systems reports significant reductions in applied forces, task com-

pletion time, and error rates when haptic feedback is present, particularly for inexperienced and novice users [2].

Commercial haptic devices remain widely used in research, training, and simulation environments due to their maturity, force-feedback fidelity, and established software support. Common examples include the 3D Systems *Geomagic Touch* and *Touch X*, the *Force Dimension omega/sigma* product range, and more recent systems such as the Haply *Inverse3 X*. These devices offer high-performance kinesthetic feedback and robust integration pathways, but are typically associated with substantially higher cost than low-cost academic prototypes and often rely on proprietary or vendor-specific software ecosystems [7].

Such constraints can limit adaptability, reduce transparency of the control loop, and complicate integration with custom simulation frameworks, experimental hardware platforms, or educational prototyping environments. This motivates interest in lower-cost and more open alternatives that preserve sufficient force-feedback capability while improving architectural transparency and extensibility.

Public vendor and reseller sources indicate that commercial systems typically occupy a substantially higher price range than academic low-cost prototypes. For example, the 3D Systems *Geomagic Touch* and *Touch X* are publicly listed at approximately US$3,400 and US$13,400 respectively [8], [9], while systems from *Force Dimension* and Haply's *Inverse3 X* are generally sold via quotation or direct sales channels [7], [10]. This places the proposed system, at approximately ~£180, substantially below the cost of established commercial alternatives.

Table 1. Indicative cost comparison of haptic interface systems from public vendor and reseller sources

System	Indicative Price / Availability
3D Systems Geomagic Touch	US$3,400
3D Systems Touch X	US$13,400
Force Dimension (omega/sigma range)	Quote on request
Haply Inverse3 X	Quote on request
Proposed system	~£180

2.2 Haptic Rendering Techniques

Research in real-time haptic simulation has advanced substantially over recent decades, particularly in collision-based interaction models. Methods such as virtual coupling and the virtual proxy (or "god-object") approach introduced by Zilles and Salisbury and later extended by Ruspini *et al.*

are widely adopted due to their robustness and favourable stability properties when interacting with rigid or constrained environments [3], [11].

These approaches rely on high servo-loop update rates, typically on the order of 1 kHz or higher, in order to achieve stable and responsive force feedback, alongside accurate end-effector pose tracking and reliable mappings from Cartesian interaction forces to actuator torques [3]. In robotic systems, such mappings are conventionally computed using the manipulator Jacobian, as formalised in operational-space control frameworks [12].

2.3 Stability in Haptic Systems

Ensuring stable haptic interaction places strict requirements on system latency, numerical integration, and control architecture. Foundational work by Adams and Hannaford demonstrated that unconditional stability in haptic systems can be achieved through the use of virtual coupling networks, which decouple device control from virtual environment dynamics and allow stability guarantees under passive user and environment assumptions [4].

These principles underpin many modern haptic rendering systems and highlight the importance of explicitly managing dynamic coupling, discrete-time implementation effects, and feedback loop timing in practical haptic interface design.

2.4 Low-Cost Haptic Hardware

Recent advances in low-cost hardware—including 3D-printed structures, compact microcontrollers, low-cost magnetic encoders, and affordable motor drive electronics—have made it increasingly feasible to construct capable actuated haptic devices using widely available components. At the same time, there is growing interest in open, modular, and reproducible research platforms that allow researchers and students to experiment with alternative control strategies, hardware layouts, and haptic interaction paradigms [1].

A prominent example is the *Hapkit* platform presented by Martinez *et al.*, which was developed as a low-cost, open-source one-degree-of-freedom haptic kit for educational applications [5]. Their later 3D-printed implementation demonstrates that a robust and customisable haptic device can be realised at a materials cost of approximately US$50, making it feasible for classroom and online deployment [5]. This work is important in showing that haptic hardware can be made accessible and

reproducible, but it is primarily focused on educational use and single-degree-of-freedom interaction.

Martinez *et al.* subsequently extended this line of work by proposing modular, open-source, customisable 3D-printed kinesthetic haptic devices that can be reconfigured into two-degree-of-freedom mechanisms, including both pantograph and serial layouts [6]. This demonstrates that low-cost educational haptic platforms can be extended toward more capable multi-DOF architectures while retaining openness and ease of fabrication.

Beyond educational hardware, Lavatelli *et al.* proposed an open-source low-cost 2-DOF haptic device intended as a mechatronic platform built largely from inexpensive and openly available components [13]. Their work is particularly relevant because it shows that low-cost two-degree-of-freedom haptic devices are feasible in an academic context, and that useful design trade-offs can be achieved between cost, simplicity, and functional haptic interaction capability.

Taken together, these studies demonstrate that low-cost and open-source haptic hardware is an active area of academic development. However, much of this work is oriented either toward educational platforms, simplified device architectures, or hardware-focused design demonstrations. Comparatively less emphasis is placed on fully integrated real-time software architectures that combine low-cost actuated hardware with transparent communication, custom haptic rendering, simulation coupling, and extensibility toward broader robotic manipulation use cases.

2.5 Summary of Literature

Overall, the literature demonstrates clear benefits of haptic feedback for robotic manipulation training and establishes well-developed theoretical foundations for stable haptic rendering. Nevertheless, there remains a practical gap in the availability of integrated systems that combine low-cost actuated hardware with transparent, architectures designed to support multiple simulation backends, suitable for educational use and rapid experimental prototyping.

3 System Architecture

3.1 System Overview

The proposed system implements a bidirectional haptic interaction loop consisting of four principal layers: the haptic device hardware, embedded control firmware, host-side simulation and haptic rendering, and graphical visualisation. A conceptual overview of the bidirectional signal and force flow across the system is shown in Fig.1.

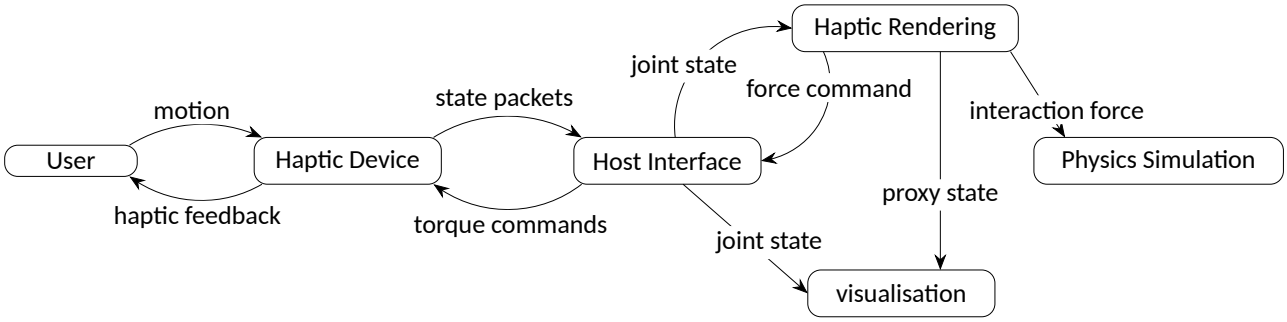


Fig. 1. High-level architecture of the proposed haptic system. User motion is measured by the haptic device and transmitted to the host interface. The host distributes state information to the haptic rendering and visualisation subsystems, while the haptic rendering module computes interaction forces, updates the simulation, and generates feedback commands that are returned to the device as actuator torque commands.

User motion is measured at the device joints and transmitted to the host simulation in real time. Based on the current virtual environment state, the host computes interaction forces which are mapped to joint torque commands and returned to the device actuators. This closed-loop structure enables simultaneous user influence on the simulated environment and force feedback from it.

To support stable real-time interaction, the system is explicitly partitioned into time-critical and non-time-critical subsystems. Low-level torque control and safety functions are executed on the embedded controller at high frequency, while collision detection, haptic rendering, physics simulation, and visualisation are performed on the host platform.

3.2 Design Constraints and Assumptions

The system architecture was developed to prioritise real-time stability, modularity, and reproducibility within the constraints of a low-cost laboratory prototype.

Stable haptic rendering typically requires servo update rates on the order of 1 kHz and minimal end-to-end latency between user motion and actuator response. Accordingly, the design assumes high-frequency embedded torque control and asynchronous host-side computation.

The user is modelled as a passive element during interaction, and the virtual environment is represented using rigid-body contact dynamics. Effects such as deformable contact, high-frequency surface textures, and frictional stick–slip behaviour are considered outside the scope of this work.

Mechanical workspace, actuator torque limits, and software safety bounds are selected to enable safe desktop operation while maintaining sufficient force capability for meaningful training interactions.

3.3 Hardware Architecture

The haptic interface is based on a planar two-degree-of-freedom mechanism providing rotational motion about two orthogonal joint axes. This configuration offers a compromise between mechanical simplicity, workspace usability, and controllable force feedback while remaining representative of common robotic manipulator interactions.

Each joint is actuated by a brushless motor operating in torque control mode and instrumented with absolute angular position sensing. The mechanical structure is designed to be lightweight and back-drivable in order to reduce reflected inertia and improve transparency of the rendered interaction forces.

3.4 Embedded Control Architecture

The embedded control layer is responsible for real-time actuator torque regulation, sensor acquisition, and bidirectional communication with the host system. Torque control is implemented using field-oriented control techniques, enabling smooth force rendering across the operating range of the device.

Joint state measurements are streamed to the host at a fixed update rate, while commanded torque references are received and applied continuously. Safety mechanisms including torque limiting and communication watchdog functions are incorporated to ensure predictable behaviour under fault conditions.

3.5 Communication Architecture

A device-agnostic packet-based communication interface is used to exchange joint state and torque command data between the embedded controller and host simulation. This abstraction enables the haptic device to be integrated with different simulation environments or future physical robotic platforms without modification to the high-level control structure.

The protocol is designed to provide deterministic timing behaviour and low transmission overhead in order to support real-time haptic interaction requirements.

3.6 Host Software Architecture

Host-side software is organised as a modular multi-threaded framework comprising subsystems for world state management, physics simulation, haptic rendering, device communication, and graphical visualisation. Separation of these components allows different update rates to be maintained for computationally intensive tasks while preserving high-frequency force computation.

A structured communication mechanism is employed to coordinate data exchange between subsystems and maintain a consistent representation of the simulated environment.

3.7 Haptic Rendering Paradigm

Interaction forces are generated using a constraint-based haptic rendering approach derived from the virtual proxy (god-object) method. In this paradigm, the unconstrained device position is coupled to a proxy constrained by the geometry of the virtual environment, enabling stable rendering of unilateral contact conditions.

A virtual coupling relationship between proxy and device states provides dynamic decoupling between the physical interface and simulated environment, supporting stable interaction under discrete-time computation.

3.8 Force Transmission to the Device

For haptic force reflection, Cartesian interaction forces computed at the virtual end-effector are mapped into joint-space actuator torques using the Jacobian transpose formulation

$$\tau = J^T(q)F \tag{1}$$

where τ is the joint torque vector, $J(q)$ is the manipulator Jacobian evaluated at joint configuration q, and F is the rendered Cartesian interaction force.

This relationship represents a quasi-static operational-space force mapping and is used specifically for force reflection from the virtual environment to the device actuators.

It does not represent the full dynamics of the haptic device, which would additionally require consideration of link inertia, Coriolis terms, damping, and actuator dynamics.

Extension toward a full dynamic operational-space formulation, following the object-space dynamics framework introduced by [12], is identified as future work.

3.9 Visualisation Layer

A graphical visualisation subsystem provides real-time feedback of the simulated environment and device interaction state. Visual rendering is intentionally decoupled from the high-frequency haptic computation loop to prevent graphical latency from degrading interaction stability.

Diagnostic overlays are incorporated to support inspection of contact conditions and system behaviour during development and testing.

4 System Implementation

This section describes the practical realisation of the proposed haptic interface system, including the mechanical construction of the device, embedded firmware architecture, communication protocol, and host-side simulation and haptic rendering software.

The implementation was designed to balance low-cost hardware selection with sufficient dynamic performance to enable stable real-time bidirectional haptic interaction.

4.1 Mechanical Design of the Haptic Device

The haptic device was realised as a planar two degree-of-freedom serial mechanism providing rotational motion about two orthogonal revolute joints. This configuration enables intuitive user interaction, allowing the end-effector to be positioned freely within a planar workspace.

A two degree-of-freedom architecture was selected in preference to higher degree-of-freedom alternatives in order to reduce mechanical complexity and minimise the number of required actuators. This was an important design consideration for maintaining the system within the project budget while still demonstrating meaningful bidirectional haptic interaction.

The structural components of the device were predominantly fabricated using additive manufacturing techniques. This enabled rapid iteration of mechanical designs and reduced moving mass, improving the perceived backdrivability of the interface.

The mechanism consists of two rigid links, with geometries and workspace illustrated in Fig. 2.

PLACEHOLDER FIGURE

Mechanical Diagram to be added here
(showing: joints,link lengths, belt reduction, motors, encoders)

Fig. 2. Placeholder for Mechanical diagram.

The link lengths were selected to provide a sufficient reachable workspace while ensuring that adequate torque could be delivered throughout the range of motion.

Actuation is provided by brushless gimbal motors (T-Motor GB2208). Due to the limited torque output of these low-cost motors, a torque amplification mechanism was required for effective operation. A GT2 timing belt transmission was therefore implemented on the base joint, providing an approximate reduction ratio of $3{:}1$. This allowed the actuator to generate sufficient torque to support both its own link and the mass of the subsequent joint and actuator.

The second joint was not equipped with a transmission stage, resulting in lower available torque at that joint. This reflects a design trade-off between mechanical complexity and performance within the constraints of the project.

Adjustable belt tensioning was incorporated via an idler pulley mechanism to reduce backlash and compensate for belt compliance. However, as discussed in Section 7.3, the transmission exhibits non-ideal behaviour under high load conditions.

Each joint rotates about an 8 mm diameter steel shaft supported by rolling element bearings to minimise friction and radial play. The corresponding links are attached via a friction fit interface within the printed components, supplemented by radial set screws to prevent slip under load.

To ensure accurate joint position measurement, absolute magnetic encoders were mounted directly on the joint axes rather than relying solely on motor-side sensing. This was particularly important for the belt-driven joint, where compliance and potential slip in the transmission would otherwise introduce errors in joint angle estimation.

PLACEHOLDER FIGURE

Final build pic to be added here

Fig. 3. Placeholder for Final build pic.

4.2 Embedded Firmware

The embedded control system is implemented using an STM32 Nucleo-G401RE development board.

Joint angular position is measured using AS5048 absolute magnetic rotary encoders communicating via SPI and mounted coaxially with the joint axes.

For the first actuator, which is coupled to the output joint through a timing belt transmission, an additional AS5600 magnetic rotary encoder is mounted on the motor shaft and interfaced via I^2C.

This encoder provides rotor position feedback for the field-oriented control loop, enabling closed-loop torque control of the brushless motor.

Encoder data are polled at a fixed sampling rate and packaged into device state packets for transmission to the host computer via a USB virtual COM interface.

The actuator control layer is implemented using SimpleFOC Mini brushless motor driver modules and the SimpleFOC library [14], selected following earlier integration challenges with alternative ESC hardware. Torque commands received from the host system are processed by the embedded firmware and applied through the SimpleFOC library, which performs the required field-oriented control computations and generates the corresponding PWM drive signals for the motors.

A communication watchdog mechanism is implemented such that if valid torque commands are not received within a predefined timeout interval, the commanded joint torques are automatically set to zero to ensure safe device behaviour.

Each actuator operates in field-oriented control (FOC) torque control mode.[15] Rotor electrical angle feedback for the FOC algorithm is obtained from the dedicated motor-shaft encoder, allowing continuous estimation of electrical rotor position and stable low-speed torque output suitable for haptic interaction.

4.3 Communication Protocol

Bidirectional communication between the host computer and the embedded controller is implemented using fixed-format binary data packets. Two primary packet types are defined: device state packets transmitted from the microcontroller to the host, and torque command packets transmitted from the host to the microcontroller.

Each packet contains a unique header identifier used for packet synchronisation, followed by joint angle or torque data and a checksum field used for basic transmission integrity verification.

Upon reception of incoming data, packet headers are validated before the payload is processed.

4.4 Host Software Architecture

The host-side software is implemented as a multithreaded C++ application composed of several modular subsystems responsible for world state management, physics simulation, haptic rendering,

device communication, and graphical rendering.

PLACEHOLDER FIGURE

Software architecture diagram to be added here
(showing threads: rendering physics haptics device comms message channels: queue vs snapshot)

Fig. 4. Placeholder for software architecture diagram.

4.4.1 Inter-Thread Communication Architecture

Communication between software subsystems executing on different threads is implemented using a structured message-passing architecture. This design avoids shared mutable state wherever possible and instead enforces controlled data exchange through explicitly defined communication channels.

A central *MessageBus* object provides named access to communication channels. Two primary channel types are implemented: queue-based command channels and snapshot-based state channels.

Queue channels are implemented using a thread-safe FIFO structure protected by a mutex. These channels are primarily used for transmitting discrete event-driven messages such as world modification commands, force application requests, or tool pose updates. Messages are published by producer threads and consumed non-blockingly by the receiving subsystem. In certain subsystems, such as the physics engine, multiple queued messages may be drained and processed in batches during a single simulation step.

Snapshot channels are used for transmitting continuously updated state information between threads in a low-latency manner. These channels implement a double-buffered data structure combined with atomic index switching to allow single-producer, multi-consumer access without requiring mutex locking during read operations. A version counter is maintained to allow consumers to detect whether a new state update has been published since the previous read.

This communication architecture enables deterministic separation between the rendering loop, physics simulation loop, haptic rendering loop, and device communication loop. By distinguishing between event-driven queue channels and continuously refreshed snapshot channels, the system reduces contention between threads while maintaining timely propagation of critical state updates.

4.4.2 World State Management

The *WorldManager* subsystem acts as the authoritative owner of the virtual environment state. It is responsible for creating, modifying, and deleting world objects, and provides a structured interface through which other subsystems issue high-level commands affecting the simulation.

Each object in the virtual world is represented by a data structure containing geometric identifiers, pose information (position, orientation, and scale), visual attributes, physical properties, and semantic role labels.

Geometric shape definitions are generated via a *GeometryFactory*, where each geometry entry represents a reusable shape description rather than a specific object instance. These entries may include signed distance field (SDF) representations used for haptic collision queries, along with rendering mesh handles and optional physics shape descriptors.

A geometry database provides indexed storage and lookup functionality for these geometry entries.

4.4.3 Rendering Subsystem

Visualisation of the simulated environment is performed by a dedicated rendering subsystem implemented using OpenGL. The renderer executes on its own thread and is responsible for displaying the current state of the virtual world, visualising haptic interaction states, and providing an interactive user interface for modifying simulation parameters at runtime.

At each render iteration, the renderer first updates the camera aspect ratio using the current framebuffer dimensions and then reads the most recent world snapshot from a snapshot communication channel. This snapshot provides a read-only view of the current simulation state, including the pose, colour, geometry identifier, role, and physical properties of each object.

For each object in the world snapshot, the renderer performs a geometry lookup through the geometry database. Each geometry entry stores a lightweight render mesh handle, which is used to retrieve the corresponding GPU-resident mesh from the render mesh registry. This separation allows simulation geometry definitions to remain independent from the underlying rendering resources.

A model matrix is constructed from the object pose using its position, quaternion orientation, and uniform scale. The object is then rendered using a lightweight *Renderable* abstraction, which combines the mesh resource, shader, object colour, and rendering options before uploading the model, view, and projection transforms to the shader and issuing the final draw call.

GPU mesh resources are represented by *MeshGPU* objects, which manage the required OpenGL vertex array, vertex buffer, and element buffer objects. In the current implementation, primitive meshes such as planes, spheres, and cubes are generated once, uploaded to GPU memory, and reused across multiple scene objects via the render mesh registry.

In addition to rendering world objects, the subsystem also visualises haptic state information received from the haptic engine. The latest haptic snapshot is consumed from a queue channel and used to render overlay markers representing the current device pose and proxy pose. This provides a useful visual indication of the relationship between the unconstrained device position and the constrained proxy position during contact interactions.

The rendering subsystem also provides interactive camera navigation. Keyboard and mouse inputs are processed through a viewport controller to allow the user to move around the virtual environment and inspect the scene from different viewpoints. Camera parameters such as field of view, near plane, far plane, yaw, and pitch are maintained as part of the renderer state.

An ImGui-based graphical user interface is layered on top of the rendered scene to support live inspection and modification of simulation parameters. Through this interface the user can select objects, adjust object pose and colour, modify physical properties such as density, damping, friction, and restitution, and create new scene objects. These edits are not applied directly to the simulation state. Instead, the UI publishes structured world modification commands to the world manager via the message bus, preserving the separation between rendering and authoritative world state ownership.

This architecture ensures that rendering remains decoupled from both the physics and haptic subsystems while still providing a responsive visual and interactive front-end for simulation control and

debugging.

4.4.4 Haptic Rendering Engine

PLACEHOLDER FIGURE

Haptic Rendering Diagram to be added here
(Show: tool position proxy position surface force vector)

Fig. 5. Placeholder for Haptic Rendering Diagram.

The haptic rendering engine is responsible for computing interaction forces between the user-controlled device and virtual objects in the simulated environment. The engine operates in a dedicated high-frequency loop and exchanges information with other subsystems using the message bus communication architecture described previously.

Real-Time Haptic Interaction Loop At each iteration of the haptic update cycle, the engine first retrieves the most recent world state snapshot using a snapshot channel. This provides the latest positions, orientations, and physical properties of all simulated objects.

The engine then drains any pending tool state messages received from the device communication subsystem. These messages contain the most recent estimate of the device end-effector pose and velocity expressed in the world coordinate frame.

Using this information, the engine performs a contact search across all relevant virtual objects. Objects designated as tool or proxy representations are excluded from this process. For each remaining object, a signed distance query is performed to determine the closest penetration condition between the tool point and the object surface.

If penetration is detected, a proxy position representing the constrained end-effector location on the object surface is computed. If no contact is present, the proxy position coincides with the current tool position.

Based on the resulting proxy and tool states, an interaction force is computed and limited to a predefined maximum magnitude. When contact occurs, an equal and opposite wrench command is generated and published to the physics simulation and device interface subsystems. A haptic state snapshot containing the current proxy pose, device pose, and rendered force is also published for use by visualisation or diagnostic components.

Proxy-Based Contact Rendering Contact handling is implemented using a proxy-based rendering approach. Each virtual object may provide a signed distance field (SDF) representation of its surface geometry. The tool position expressed in the world frame is first transformed into the local coordinate frame of each object under consideration.

A signed distance value ϕ and corresponding surface gradient are then obtained from the SDF. A negative signed distance indicates penetration of the tool point into the object volume. In this case, the tool position is projected onto the closest point on the surface along the direction of the SDF gradient. This projection is performed in the local object frame and subsequently transformed back into world coordinates to define the proxy position.

This mechanism allows arbitrary implicit or mesh-derived geometries to be rendered haptically using a unified contact model.

Virtual Coupling Force Model Interaction forces are generated using a virtual coupling model based on a linear spring–damper relationship between the proxy position and the actual device position. The rendered Cartesian force is given by

$$\mathbf{F} = K\left(\mathbf{x}_p - \mathbf{x}_t\right) + D\left(\mathbf{v}_p - \mathbf{v}_t\right)$$

where $\mathbf{x}_p$ and $\mathbf{x}_t$ denote the proxy and tool positions respectively, and $\mathbf{v}_p$ and $\mathbf{v}_t$ represent the corresponding velocities.

The proxy velocity is estimated using finite differences between successive proxy poses, while the tool velocity is obtained from the incoming device state message. In the present implementation the stiffness parameter is set to $K = 2000\,\mathrm{N\,m^{-1}}$ and an equivalent virtual mass parameter of $M = 0.2\,\mathrm{kg}$ is used to compute the damping coefficient according to

$$D = 0.7 \cdot 2\sqrt{KM}.$$

To ensure safe and stable interaction behaviour, the magnitude of the rendered force is limited to a maximum allowable value of 15 N.

Coupling to the Physics Simulation When contact with a virtual object is detected, the haptic engine generates a wrench command representing the interaction force applied at the contact point. This wrench is published to the physics simulation subsystem, which applies the equal and opposite force to the corresponding dynamic actor during the next physics update step.

This bidirectional coupling ensures that user interaction influences the motion of simulated objects while simultaneously providing force feedback to the user. In addition, the haptic engine publishes a structured snapshot message containing the current device pose, proxy pose, and rendered force, enabling other subsystems such as rendering or logging modules to access the current interaction state.

4.4.5 Physics Simulation Integration

Physical behaviour of virtual objects is simulated using the NVIDIA PhysX rigid body dynamics engine. The physics engine operates as a separate subsystem that advances the dynamic state of objects based on applied interaction forces and predefined physical parameters.

At each external simulation update, the physics engine first checks whether the topology or physical properties of the virtual world have been modified. If changes are detected, all rigid actors are rebuilt from the current authoritative world state representation. This ensures consistency between the physics scene and the higher-level world model.

Interaction forces generated by the haptic rendering engine are communicated to the physics engine via a queue-based message channel. These messages contain a wrench specification including the target object identifier, applied force vector, point of application in world coordinates, and an interaction duration. In the present implementation, forces are converted into impulses using the relation

$$\mathbf{J} = \mathbf{F}\,\Delta t$$

and applied to the corresponding rigid body using PhysX impulse-based force application methods.

The physics engine advances the simulation using a fixed internal timestep. An accumulator-based integration strategy is employed, whereby incoming variable-duration external timesteps are accumulated until sufficient time has elapsed to perform one or more fixed simulation steps. This approach improves numerical stability and ensures deterministic solver behaviour.

Dynamic rigid bodies are configured with linear and angular damping parameters obtained from the world model. Continuous collision detection (CCD) is enabled to improve robustness under fast contact interactions. Solver iteration counts are explicitly configured to balance computational cost against constraint resolution accuracy.

Following each simulation update, the resulting poses of dynamic actors are written back into the authoritative world state maintained by the world management subsystem. This write-back mechanism ensures that subsequent rendering, collision queries, and haptic force computations operate on a consistent and up-to-date representation of the simulated environment.

4.4.6 jacobians maybe

4.4.7 Logging Framework

A dedicated logging and diagnostics subsystem was implemented to support timing analysis, communication validation, and offline performance evaluation without perturbing the real-time device control loop.

To avoid introducing disk I/O latency into the 1 kHz communication thread, logging was implemented as an asynchronous message-passing pipeline using dedicated channels on the message bus.

Two structured log message types were defined:

- device state logs, capturing all successfully parsed incoming device packets
- device timing logs, capturing matched receive–compute–transmit control cycles

These messages include host-side timestamps, device sequence counters, MCU timestamps, measured joint angles, rendered forces, and transmitted torque commands.

A dedicated background logging thread continuously drains the logging channels and appends entries to in-memory buffers. At application shutdown, the captured data are exported to CSV files

for offline analysis.

This architecture ensures that instrumentation overhead does not affect the timing-critical haptic control path.

5 Experimental Methodology

This chapter describes the experimental procedures used to evaluate the communication performance, closed-loop timing behaviour, simulation fidelity, and safety behaviour of the proposed haptic rendering system.

The evaluation was designed to assess whether the implemented architecture satisfies the real-time requirements necessary for stable haptic interaction, with particular focus on communication reliability, timing determinism, and force-feedback safety.

5.1 Communication Performance Tests

To evaluate serial communication performance independently of the main application, a dedicated command-line test utility was developed. The tool provided two operating modes: a packet rate and integrity mode for analysing streamed device packets, and a ping–echo mode for measuring round-trip latency.

In both modes, incoming serial data were processed using a buffered parser with explicit packet header search, checksum validation, and byte-wise re-synchronisation to ensure robustness to fragmented or corrupted input. Test outputs were written to CSV files for offline statistical analysis, including effective receive rate, inter-arrival jitter, packet drops, out-of-order events, and RTT percentile metrics.

The communication tests were conducted across a range of target transmission rates from 100 Hz to 2000 Hz in order to assess the effect of increasing packet frequency on communication reliability and timing performance.

5.1.1 Packet Integrity and Rate Tests

Packet integrity and communication timing were evaluated using the dedicated rate-test mode of the communication test utility.

For each target transmission frequency, the firmware was configured to stream fixed-format device state packets at a constant rate. Tests were performed at nominal frequencies of 100 Hz, 250 Hz, 500 Hz, 1000 Hz, 1250 Hz, and 2000 Hz.

For each test condition, the host continuously received and parsed incoming packets over the USB virtual COM interface. The following performance metrics were recorded:

- valid packet count
- dropped packets inferred from sequence gaps
- out-of-order packets
- host-observed inter-arrival time
- effective receive rate
- inter-arrival jitter

All results were exported to CSV format for offline statistical analysis and visualisation.

5.1.2 Round-Trip Latency Test

Round-trip latency was evaluated using a ping–echo communication test.

In this test, the host transmitted timestamped ping packets to the embedded controller, which immediately returned an echo packet containing the original sequence number and timestamp.

Tests were performed using transmission intervals of 1 ms, 2 ms, and 10 ms.

For each test, the following metrics were computed:

- mean RTT
- median RTT
- 95th percentile RTT
- 99th percentile RTT
- minimum and maximum RTT

- timeout count

These results were used to characterise communication latency and timing consistency under different transmission loads.

5.1.3 End-to-End Host-Side Timing Test

In addition to the standalone serial communication tests, end-to-end timing of the host-side closed-loop haptic pipeline was evaluated using a dedicated runtime logging framework integrated into the main application.

During normal operation of the haptic system, the device communication thread published structured timing and state log messages to a dedicated logging subsystem. To avoid perturbing the real-time control path, log messages were transferred asynchronously to a separate logging thread and buffered in memory during execution. After the application was stopped, the captured logs were exported to CSV files for offline analysis in MATLAB.

Two primary log files were generated. The first, `device_timing.csv`, contained matched state-command timing records for closed-loop control cycles, including host-side timestamps associated with packet parsing, tool-state publication, wrench consumption, and torque command transmission. The second, `device_state_log.csv`, contained a record of every valid parsed incoming state packet, including host receive timestamps, packet sequence identifiers, MCU timestamps, and measured joint angles.

From these logs, the following metrics were evaluated:

- host-side end-to-end latency between state-packet parsing and torque command transmission
- intermediate host pipeline delays between communication, haptic processing, and transmission stages
- matched state-command sequence consistency
- parsed state-packet inter-arrival timing on the host
- sequence gap statistics for identifying skipped packets
- estimated parsed packet rate and MCU-side timing trends

Because the host and microcontroller clocks were not synchronised, the analysis was restricted to host-side timing differences and trend-based interpretation of MCU timestamps. Absolute one-way

latency between host and device could therefore not be inferred directly.

5.2 Simulation Validation

Simulation validation tests were designed to assess both the correctness of the haptic rendering model in isolation and the stability of the integrated system during interaction.

Two complementary forms of evaluation were performed. First, the behaviour of the virtual coupling and contact model was assessed by observing the computed force output of the simulation under controlled interaction conditions. Second, the full system, including the physical device, communication pipeline, and actuator feedback, was evaluated during sustained user interaction.

5.2.1 Virtual Wall Test

A virtual wall test was performed to validate the basic unilateral contact behaviour of the haptic rendering engine. In this test, a simple planar contact boundary was defined in the virtual environment and the device end-effector was manually moved toward and against the wall during repeated interaction trials.

The purpose of this test was to verify that the proxy-based rendering method correctly prevented penetration of the constrained proxy through the wall surface while allowing the unconstrained device position to continue moving, thereby generating a restoring virtual coupling force.

The test was used to assess both the computed force response of the model and its behaviour when coupled to the physical device.

The following behaviours were examined:

- absence of significant rendered force when the end-effector remained outside the wall
- increase in rendered force as penetration depth increased
- qualitative agreement between the observed force response and the implemented spring–damper coupling model

5.2.2 Constrained Motion Along a Virtual Surface

In addition to evaluating normal contact behaviour, the system was tested for its ability to support constrained motion along a virtual surface.

During this test, the device end-effector was brought into contact with a planar virtual wall and allowed to move freely along the surface while maintaining contact. This behaviour arises naturally from the proxy-based rendering method, in which motion normal to the surface is resisted while tangential motion remains unconstrained.

The following aspects were assessed qualitatively:

- ability to maintain contact with the surface while moving along it
- absence of significant resistance to tangential motion
- stability of motion during sustained sliding contact
- smoothness of transitions between free-space and constrained motion

This test demonstrated that the system could provide implicit motion guidance along surfaces, which is relevant for tasks involving contact-based manipulation and training scenarios.

5.3 Safety Validation

Safety validation tests were conducted to ensure that the system remained bounded and behaved predictably under both normal and fault conditions. These tests focused on verifying the effectiveness of software and embedded safety mechanisms, including torque saturation, rate limiting, and watchdog-based fail-safe behaviour.

5.3.1 Torque Saturation Test

A torque saturation test was performed to verify that actuator command limits were correctly enforced across both the host and embedded control layers.

Force outputs generated by the haptic rendering engine were limited on the host side to a predefined maximum value. In addition, the embedded controller applied a secondary torque clamp to ensure that actuator commands remained within safe operating bounds regardless of incoming data.

During testing, interaction scenarios producing large virtual forces were introduced, and the resulting actuator commands were observed to confirm that:

- force commands were correctly limited on the host side
- transmitted torque commands were further bounded by the embedded controller
- actuator output remained within safe and predictable limits under all conditions

This layered saturation approach provided robustness against both software and communication-related anomalies.

5.3.2 Rate Limiting Test

A rate limiting test was conducted to evaluate the effect of limiting the rate of change of commanded torque at the embedded controller level.

A slew-rate limiter was implemented on the microcontroller to constrain the maximum rate at which actuator torque commands could change. This was introduced to reduce high-frequency oscillations and improve stability during rapid contact transitions.

During testing, interaction scenarios involving abrupt changes in virtual force were generated, and the resulting actuator behaviour was observed to determine whether:

- rapid command changes were smoothed into bounded torque ramps
- high-frequency oscillations were reduced during contact events
- overall system stability was improved during interaction

This mechanism was introduced as a stability enhancement to mitigate oscillatory behaviour in the haptic feedback loop.

5.3.3 Watchdog Timeout Test

A watchdog timeout test was performed to verify safe system behaviour under loss of communication between the host and embedded controller. In this test, the stream of valid incoming torque commands was intentionally interrupted during operation.

The response of the embedded controller was then observed to confirm that actuator torque commands were reset to zero after the predefined timeout interval.

The following conditions were checked:

- whether the watchdog triggered reliably when command updates ceased
- whether commanded torque was removed within the intended timeout window
- whether the device returned to a safe passive state after communication loss

This test was used to validate the fail-safe behaviour of the embedded control layer in the event of communication interruption or host-side failure.

5.4 Hardware Validation

Hardware validation tests were performed to assess the behaviour of the physical actuation system and the overall device during interaction. These tests focused on qualitative evaluation of actuator responsiveness and the stability of sustained user interaction with the haptic interface.

5.4.1 Actuator Torque Response Characterisation

Basic actuator response behaviour was assessed using a standalone motor test prior to full system integration. In this test, step changes in commanded torque were applied to an individual actuator, and the resulting mechanical response was evaluated qualitatively.

The purpose of this test was to verify that the motor and control hardware could produce responsive and stable torque output suitable for subsequent use in the haptic interface.

The evaluation focused on:

- qualitative responsiveness to changes in commanded torque
- presence of oscillatory or unstable behaviour
- smoothness of the resulting motion and resistive feel
- repeatability across repeated command inputs

The assessment was performed through direct observation and manual interaction with the actuator, and therefore represents a qualitative validation of actuator behaviour rather than a precise quantitative characterisation.

The behaviour observed during this standalone test was consistent with the actuator performance required for stable force rendering in the integrated haptic system.

5.4.2 Stable Contact Test

A stable contact test was conducted to evaluate the behaviour of the full closed-loop system during sustained interaction with a virtual surface.

In this test,the device end-effector was brought into contact with a virtual object and maintained contact while observing the resulting force feedback behaviour.

Unlike the virtual wall test, which focuses primarily on validating the contact model, this test evaluates the stability and usability of the integrated system under continuous interaction conditions.

The following aspects were examined:

- ability to maintain continuous contact without runaway motion
- presence of oscillation, chatter, or instability
- consistency of the rendered resistive force
- qualitative usability of the interaction

This test served as an integrated validation of the practical haptic stability of the prototype under representative user interaction conditions.

6 Results

This chapter presents the results obtained from the communication, simulation, safety, and hardware validation tests described in Chapter 5. The results are used to assess whether the implemented system achieved sufficiently reliable and stable real-time performance for low-cost haptic interaction.

Particular attention is given to packet timing behaviour, round-trip communication latency, host-side closed-loop timing, and qualitative stability and force feel during sustained interaction with virtual constraints.

6.1 Communication Performance

Communication performance was evaluated using both standalone serial tests and runtime logging of the integrated haptic application. The standalone tests were used to characterise packet delivery

behaviour and round-trip latency under controlled conditions, while the runtime logs were used to assess host-side timing behaviour of the closed-loop haptic pipeline during normal operation.

6.1.1 Packet Rate and Integrity Results

The results presented in this section correspond to the packet integrity and rate tests described in Section 5.1.1.

Packet rate testing showed that communication performance remained stable over a wide range of requested transmission frequencies, although the behaviour was not uniform across all tested rates.

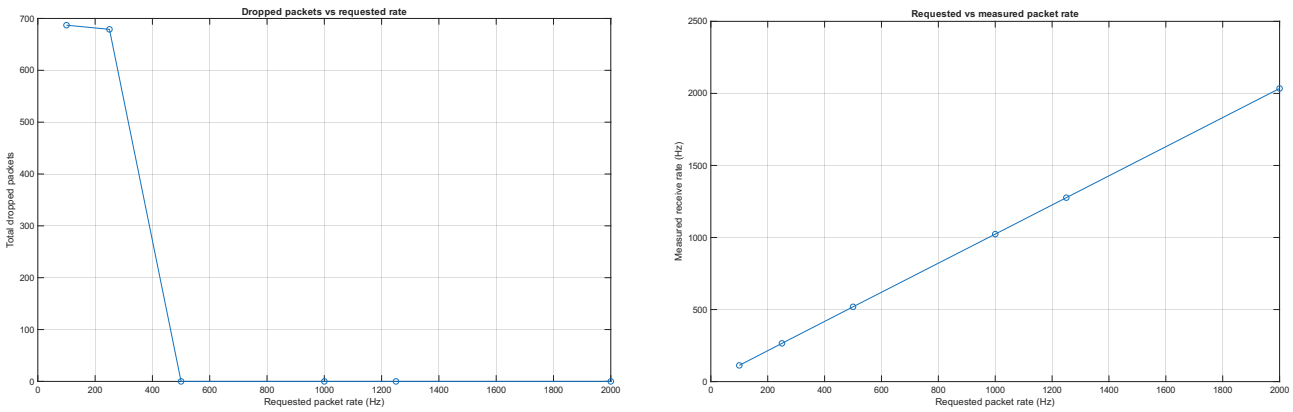
At lower requested rates of 100 Hz and 250 Hz, packet loss was observed, with drop fractions of approximately 20.3% and 8.5% respectively. In contrast, for requested rates of 500 Hz and above, no dropped packets were observed in the recorded data, as shown in Fig. 6(a), where packet loss occurs only at the lowest transmission rates. Out-of-order packets were extremely rare across all higher-rate tests, with only a single out-of-order event recorded in each case.

Interestingly, packet loss was more pronounced at lower transmission rates. This may be attributed to buffering and scheduling effects in the host or serial driver, where lower rates reduce the likelihood of continuous buffer flushing.

The measured receive rate tracked the requested transmission rate closely across the full test range. As shown in Fig. 6(b), the relationship between requested and measured rate was approximately linear, indicating that the communication system maintained throughput proportional to the commanded transmission frequency.

Inter-arrival jitter, defined in Section 5.1.1 as the standard deviation of packet inter-arrival time, was present in all tests, with standard deviation decreasing as the requested rate increased, as shown in Fig. 7. However, percentile analysis revealed that occasional large timing excursions persisted even at higher rates, indicating the presence of intermittent latency spikes. This suggests that while average jitter decreases with increasing rate, worst-case timing behaviour remains non-negligible, which is relevant for haptic stability.

Overall, these results indicate that the serial communication link is capable of supporting packet rates at or above 1 kHz with negligible packet loss in the tested configuration. This is sufficient to



(a) Dropped packets **(b)** Rate tracking

Fig. 6. Communication performance across transmission rates. (a) Total dropped packets as a function of requested rate, showing packet loss only at low frequencies. (b) Measured receive rate versus requested transmission rate, demonstrating approximately linear tracking.

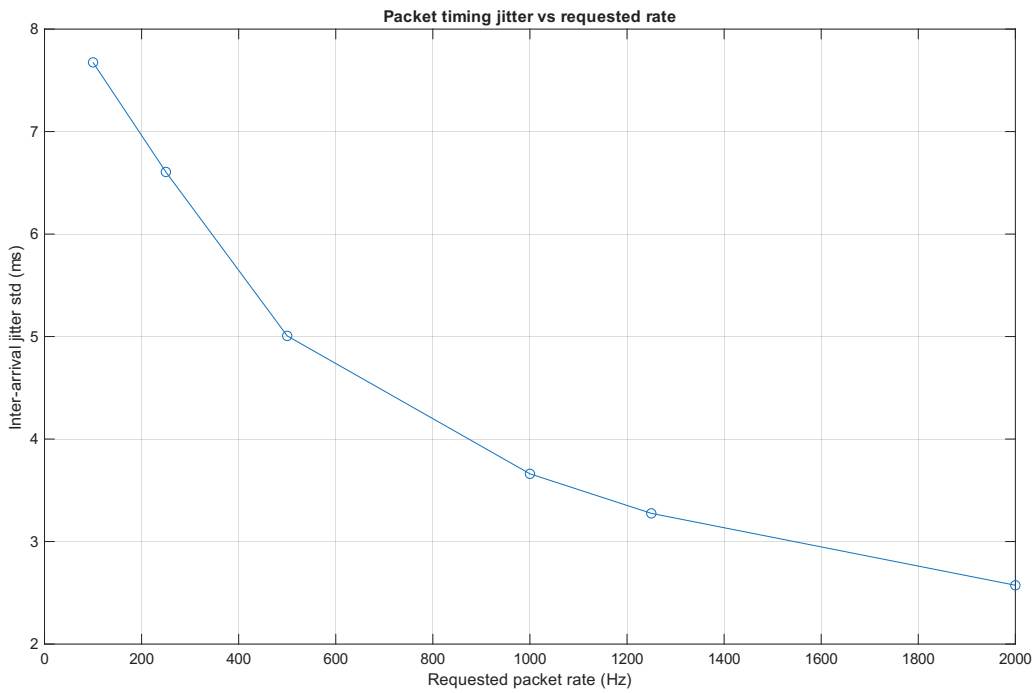


Fig. 7. Standard deviation of packet inter-arrival time as a function of requested transmission rate. Jitter decreases with increasing rate, although occasional large timing excursions remain present.

support stable haptic feedback loops operating at approximately 1 kHz, a typical requirement for high-fidelity force rendering.

6.1.2 Round-Trip Latency Results

The following results correspond to the round-trip latency test described in Section 5.1.2.

Round-trip latency results were highly consistent across all tested ping intervals. For nominal transmission intervals of 1 ms, 2 ms, and 10 ms, the success rate was 100% in all cases, with no timeouts

observed over 1000 samples per test.

The mean measured round-trip latency remained approximately constant across all three test conditions, with values of 15.60 ms, 15.64 ms, and 15.67 ms respectively. Similarly, the 95th and 99th percentile RTT values were tightly grouped around 16.2–16.4 ms, indicating stable and predictable timing behaviour across repeated measurements. This behaviour is further illustrated in Fig. 8, where the cumulative distributions for all three tests overlap closely and rise sharply at approximately 15–16 ms, indicating that RTT is largely independent of the selected ping interval.

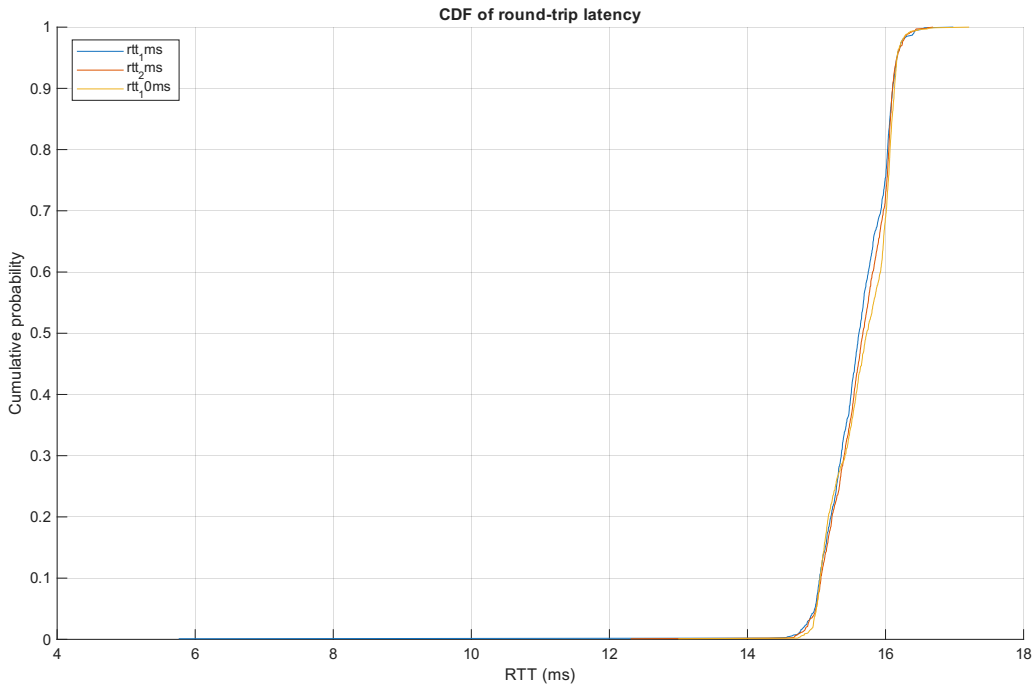


Fig. 8. Cumulative distribution functions of measured round-trip latency for nominal ping intervals of 1 ms, 2 ms, and 10 ms. The distributions overlap closely and rise sharply at approximately 15–16 ms, indicating that round-trip latency is dominated by a largely fixed delay component rather than by the selected ping interval.

The standard deviation of RTT remained below 0.52 ms in all cases, suggesting relatively low variability in round-trip timing. Minimum and maximum observed RTT values showed occasional larger deviations from the typical latency range, particularly in the 1 ms test, but these did not significantly affect the overall distribution.

These results indicate that round-trip communication delay in the tested system is dominated by a relatively fixed latency component rather than by the selected transmission interval. In other words, increasing the ping frequency does not significantly reduce the measured latency, implying that the dominant delay arises from the serial communication stack, USB virtual COM interface, or host-side scheduling behaviour.

From a system perspective, this indicates that while the system is capable of sustaining high up-

date rates with low host-side processing delay, overall responsiveness is limited by communication latency rather than computational performance. In particular, the observed round-trip latency of approximately 15–16 ms is significantly larger than the nominal 1 ms update period associated with 1 kHz haptic rendering, meaning that communication delay dominates the closed-loop response. As a result, achievable virtual stiffness is constrained more strongly by communication delay than by control loop execution speed.

Nevertheless, the low variability and high reliability of the measured RTT indicate that the delay is predictable, which is beneficial for maintaining stability in the haptic feedback loop when combined with appropriate damping and rate-limiting strategies.

6.1.3 End-to-End Host-Side Timing Results

These results correspond to the end-to-end host-side timing test described in Section 5.1.3.

End-to-end timing behaviour of the integrated haptic pipeline was evaluated using runtime logging of matched state-command cycles and raw parsed state packets.

Analysis of `device_timing.csv` showed that all recorded control cycles were successfully matched, with a matched fraction of 1.000, indicating consistent correspondence between received state packets and transmitted torque commands.

The host-side end-to-end latency, defined as the time between state-packet parsing and completion of torque command transmission, was found to be low. The mean latency was 0.299 ms, with a median of 0.104 ms. The 95th percentile latency was 0.546 ms, and the 99th percentile was 2.04 ms, with a maximum observed latency of 3.35 ms. As shown in Fig. 9, the distribution was concentrated at low values, with prominent peaks near 0 ms and around 0.5 ms, and only a limited number of higher-latency events.

Breakdown of intermediate pipeline delays showed that the majority of this latency was associated with serial transmission. The mean transmission duration was 0.249 ms, while the delay associated with publishing tool state and processing the haptic pipeline remained small, with mean values of 0.024 ms and 0.0044 ms respectively. This indicates that host-side computation and message-passing overhead were negligible relative to communication time.

Sequence analysis showed that command sequence progression remained consistent, with a mean command sequence gap of 1.14 and a maximum of 6, indicating that command updates were gen-

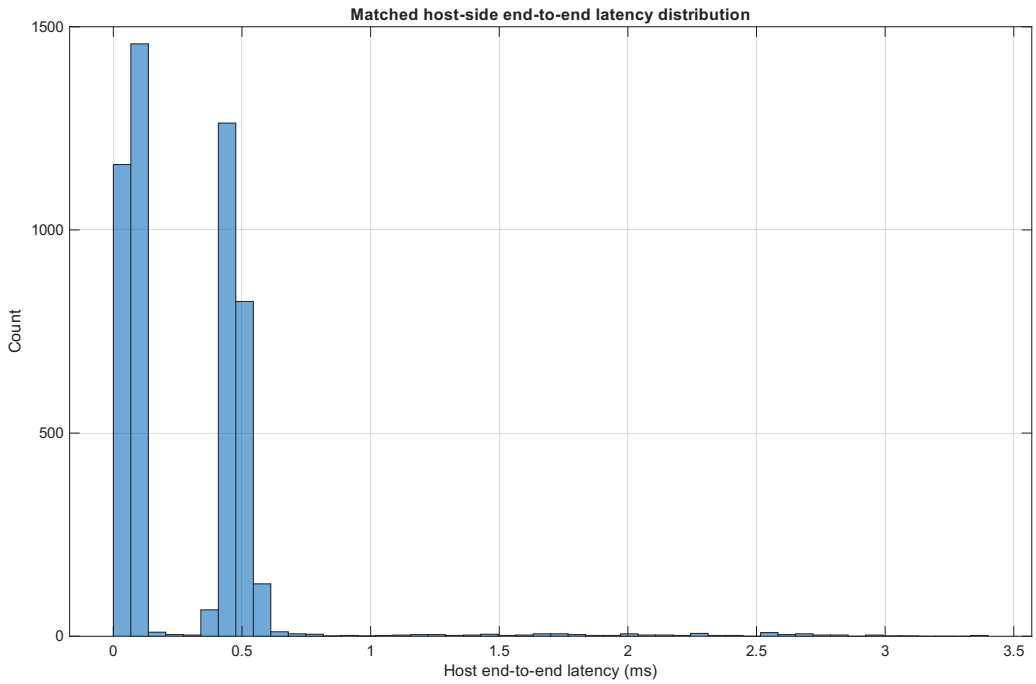


Fig. 9. Distribution of matched host-side end-to-end latency, measured from state-packet parsing to completion of torque command transmission. The latency is concentrated at low values, with prominent peaks near 0 ms and around 0.5 ms, and only a small number of higher-latency outliers.

erally applied sequentially with minimal skipping. In contrast, the matched state sequence gap was significantly larger, with a mean of 33.45. This reflects the fact that the control loop operated on a subset of the available incoming state packets rather than processing every parsed packet. Here, sequence gap denotes the difference between consecutive sequence identifiers, such that a value of 1 corresponds to consecutive packets or commands and larger values indicate skipped intermediate sequence numbers.

A total of 54 161 valid state packets were parsed during the test, compared to 5 048 matched control cycles, confirming that the control loop operated on a subset of the incoming packet stream rather than consuming every parsed packet. The estimated parsed packet rate, computed from the mean host-side inter-arrival time, was approximately 1006 Hz, which is close to the expected 1 kHz transmission rate.

Analysis of host-side parsed-packet inter-arrival timing revealed significant variability. The mean inter-arrival time was 0.994 ms, but the median was only 0.0016 ms, reflecting that many packets were processed in immediate succession within bursts. The large difference between mean and median inter-arrival time indicates that packets were frequently processed in rapid bursts following periods of delay, rather than arriving at uniform intervals. This behaviour is illustrated in Fig. 10, where a dominant peak near 0 ms is accompanied by a much smaller secondary concentration around 14–16 ms. The 95th and 99th percentile inter-arrival times were 14.5 ms and 15.8 ms re-

spectively, confirming the presence of intermittent host-side stalls.

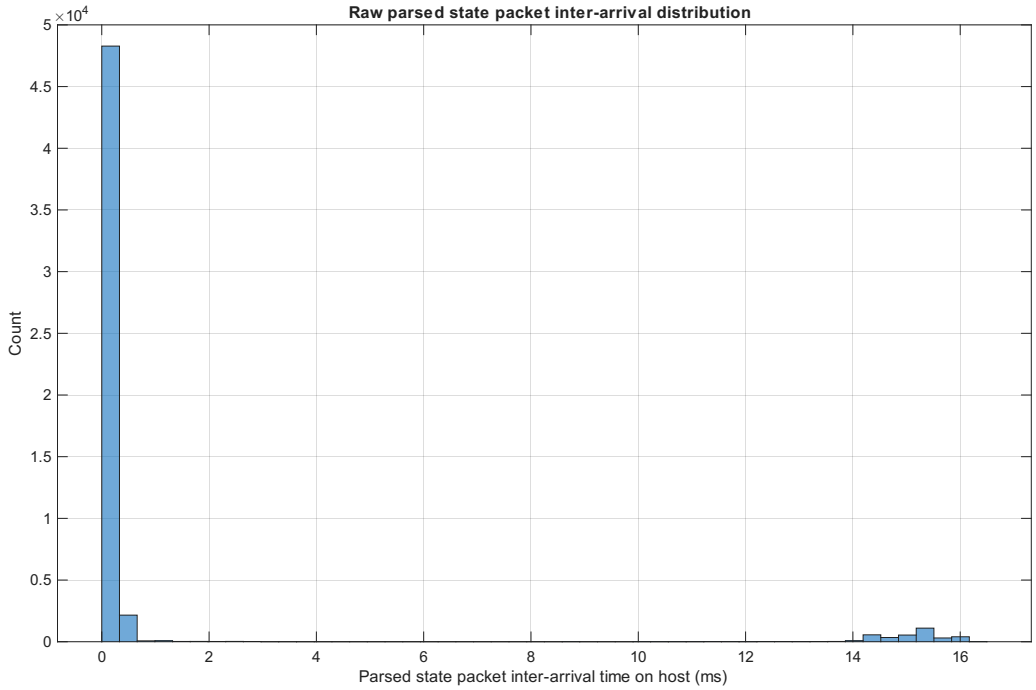


Fig. 10. Distribution of host-side inter-arrival time between consecutively parsed state packets. A dominant peak near 0 ms and a smaller concentration around 14–16 ms indicate burst-like packet processing at the host, consistent with buffered delivery and intermittent scheduling delays.

Despite this non-uniform host-side processing pattern, analysis of MCU-side timestamps showed that the embedded system maintained a stable and consistent update rate. The effective MCU period per state was approximately 1.00 ms, corresponding to an update rate of 1000 Hz, with very low variability.

Taken together, these results indicate that while packet generation on the embedded controller was highly regular, packet processing at the host occurred in a non-uniform burst-like pattern. These observations strongly suggest that buffering and scheduling effects within the USB virtual COM interface and host operating system are the primary causes of the observed burst-like packet delivery behaviour, although the current instrumentation cannot fully separate chunk delivery timing from parser timing.

Importantly, the host-side processing pipeline was sufficiently fast to handle this behaviour, resulting in low end-to-end latency and stable closed-loop operation. Overall, the integrated system achieved high update rates with low host-side processing latency despite non-uniform packet processing at the host. This supports the feasibility of the proposed architecture for real-time haptic interaction, with the primary limitation arising from communication latency rather than host-side computation, as also reflected in the round-trip latency measurements.

6.2 Simulation Validation Results

Simulation validation tests were conducted with actuator output disabled, allowing the behaviour of the haptic rendering model to be evaluated independently of hardware dynamics and communication effects.

6.2.1 Virtual Wall Model Response

A virtual wall test was performed to evaluate the output forces generated by the haptic rendering engine in isolation from physical feedback.

The rendered force exhibited the expected unilateral contact behaviour. No significant force was produced when the end-effector remained outside the virtual boundary, while penetration resulted in a restoring force that increased with penetration depth, consistent with the implemented spring–damper virtual coupling model. This is consistent with the expected response of a unilateral contact model, where force is applied only during penetration and is directed along the surface normal.

However, this behaviour degraded under conditions of large penetration. In these cases, the magnitude and direction of the rendered force became inconsistent. This is attributed to limitations of the projection-based contact model, where the closest-point computation becomes less stable far from the surface. As a result, discontinuities in the estimated surface normal and projection point can occur, leading to reduced or poorly directed restoring forces.

This limitation is particularly relevant for haptic stability, as discontinuities in the estimated contact normal can introduce non-physical force variations, potentially degrading perceived stiffness and smoothness during interaction.

This test confirms that the contact model produces physically consistent behaviour under typical interaction conditions, while also highlighting limitations under extreme penetration scenarios.

6.2.2 Constrained Motion Model Behaviour

The behaviour of constrained motion was also evaluated with actuator output disabled.

When the end-effector was moved along the virtual wall surface, the computed force remained predominantly normal to the surface, resisting motion into the wall while allowing tangential motion. This behaviour is characteristic of virtual fixture or constraint-based haptic rendering, where motion is restricted in selected directions while remaining unconstrained in others. Such behaviour is particularly relevant for training applications, where users must learn to follow constrained paths or avoid forbidden regions.

These results demonstrate that the simulation model can reliably generate constraint forces aligned with surface geometry, supporting its suitability for guidance and training-based interaction tasks.

6.3 Safety Validation Results

The safety mechanisms were evaluated primarily as functional pass/fail protections, since their purpose was to enforce bounded behaviour under fault or extreme-input conditions rather than to optimise a continuous performance metric.

6.3.1 Torque Saturation and Rate Limiting

The combined torque saturation and rate limiting mechanisms were effective in maintaining stable and safe actuator behaviour.

Under conditions where large forces were generated by the haptic rendering engine, the host-side force clamp limited the magnitude of commanded forces before transmission. This was further reinforced by an embedded torque clamp, ensuring that actuator outputs remained within predefined safety bounds.

In addition, the implemented slew-rate limiter on the microcontroller significantly improved stability during interaction. Rapid changes in commanded torque were smoothed into gradual transitions, reducing high-frequency oscillations and improving the perceived stability of contact.

Without rate limiting, abrupt force changes resulted in noticeable vibration and unstable behaviour. The inclusion of the rate limiter reduced these effects and contributed to stable sustained interaction.

Overall, the layered safety approach of force clamping and torque rate limiting was found to be essential for achieving stable and controllable haptic feedback.

6.3.2 Watchdog Timeout Behaviour

The watchdog was triggered consistently when command updates were absent for longer than the configured timeout interval of X ms.

When the stream of incoming torque commands from the host was interrupted, the embedded controller detected the absence of updates and reset actuator torque commands to zero after the predefined timeout interval.

This resulted in the device returning to a passive, unpowered state without exhibiting unintended motion or sustained force output.

The watchdog behaviour was observed to be consistent and repeatable across multiple trials, providing a robust safeguard against communication failure or host-side faults.

These results confirm that the embedded safety mechanisms are effective in preventing unsafe actuator behaviour in the presence of communication disruption.

6.4 Hardware Validation Results

Hardware validation focused on qualitative assessment of responsiveness, stability, and perceived interaction behaviour during representative haptic tasks. Direct end-effector force measurement was not available in the prototype, so hardware evaluation was based primarily on observed stability and perceived interaction behaviour rather than absolute force accuracy.

In addition, actuator control was implemented using field-oriented control without current sensing, meaning that commanded torque could not be treated as a precise or directly calibrated measure of physical output torque. The device therefore conveyed qualitative variations in force and approximate relative magnitude, rather than accurately scaled physical force output.

It should also be noted that, although the device was designed as a 2-DOF interface and both joints remained available for device-to-host position sensing, final force-feedback testing was conducted with actuation available only on the first joint. This was due to the failure of one actuator during development, which could not be replaced within the project budget. As a result, the hardware results reported here reflect partial force-feedback operation of the intended 2-DOF architecture rather than full two-axis haptic rendering.

6.4.1 Actuator Torque Response Characterisation

Standalone testing of individual actuators demonstrated that the motor and control hardware were capable of producing responsive and stable torque output.

Step changes in commanded torque resulted in immediate and clearly observable changes in actuator behaviour. Although the actuator control did not provide a calibrated one-to-one relationship between commanded and physical torque, the response was generally smooth and repeatable, with limited overshoot and no sustained oscillatory instability under typical operating conditions.

Some minor oscillations were observed at higher torque levels or during rapid command changes; however, these were mitigated through the use of torque rate limiting and appropriate control parameter selection.

Overall, the qualitative assessment indicated that the actuator response was sufficiently fast and stable for use in haptic rendering applications, where timely and predictable force output is required.

6.4.2 Stable Contact Behaviour (Virtual Wall Interaction)

This test represents the physical realisation of the virtual wall interaction previously validated in simulation.

The integrated system was able to support sustained contact interaction without loss of stability.

During testing, continuous contact with a virtual surface could be maintained for extended periods without runaway motion or divergence. The rendered force remained consistent, and the interaction was perceived as stable and controllable.

Some low-level vibration and noise were present during contact, particularly under high stiffness settings. However, this did not lead to instability and was reduced through the combined use of damping and torque rate limiting.

The system remained responsive to user input while maintaining a stable resistive force, indicating that the closed-loop architecture is suitable for practical haptic interaction scenarios.

These results demonstrate that the prototype can achieve stable contact interaction under real-time conditions, despite communication variability and non-uniform packet arrival behaviour.

It is noted that the large penetration effects observed during simulation validation were less prominent during physical interaction. In practice, the rendered contact force limits penetration depth by opposing user motion. However, due to the finite torque output of the actuators and the achievable virtual stiffness, some penetration still occurs during contact.

This effect is more pronounced when interacting with small or thin virtual objects, where relatively small physical displacements may correspond to significant penetration in the virtual model. As a result, the limitations of the projection-based contact formulation may still influence force consistency under certain conditions.

Although stable contact behaviour was achieved, the rendered force did not exhibit perfectly rigid characteristics. The interaction was observed to have a degree of compliance, with force increasing gradually rather than instantaneously. This behaviour reflects limitations in actuator torque capability, control fidelity, and transmission behaviour, which are discussed further in Section 7.3.

Under attempts to push strongly through a rigidly rendered object, transmission non-idealities were also observed at the base joint. In particular, belt slip occurred under sufficiently high load, leading to a reduction in perceived resistance and a loss of consistency in the rendered force during extreme contact conditions. Although this behaviour was not dominant during typical interaction, it became apparent during aggressive push-through attempts and further limited the maximum practically renderable stiffness.

6.4.3 Stable Contact Behaviour (Constrained Motion)

Constrained motion behaviour was also evaluated under physical operation of the device. Given that physical actuation was available only on the first joint during final testing, the constrained-motion behaviour observed in hardware represents a partial implementation of the intended two-axis guidance behaviour.

When the end-effector was brought into contact with a virtual surface and moved along it, the system generated forces predominantly normal to the surface, resisting penetration while allowing motion tangential to the constraint. This resulted in guidance-like behaviour consistent with the constrained motion model validated in simulation.

An additional observation was made when the device was released near a virtual surface. When positioned close to the surface, the end-effector could remain supported by the rendered contact

force, behaving similarly to a physical object resting against a rigid boundary. In contrast, when released further from the surface, the end-effector moved freely until contact was re-established, at which point motion became constrained along the surface.

This behaviour demonstrates that the system is capable of producing physically intuitive constraint forces, while also reflecting the finite stiffness and force limits of the actuator system. The transition between free motion and constrained interaction was smooth and did not result in instability.

The guidance behaviour was effective but not perfectly rigid, with some compliance and variation in constraint force observed during motion. This was consistent with the limited actuator stiffness and partial actuation available in the final hardware configuration.

These results confirm that the prototype can produce stable and physically intuitive constraint interactions under real-time conditions, despite communication variability and actuator limitations.

6.5 Summary of Results

Overall, the results demonstrate that the proposed system is capable of achieving stable and reliable real-time haptic interaction within the constraints of a low-cost hardware platform.

Communication performance supports high update rates with low packet loss, although a fixed round-trip latency of approximately 15–16 ms imposes a fundamental limitation on achievable responsiveness and virtual stiffness. Despite this, host-side processing latency remains negligible, and the system maintains stable closed-loop behaviour under realistic operating conditions.

Simulation validation and partial hardware validation confirm that the haptic rendering model produces physically consistent contact and constraint forces, while embedded safety mechanisms ensure robust and predictable behaviour under fault conditions. However, the hardware results also show that the rendered interaction is not perfectly rigid: actuator torque limits, partial actuation, and transmission non-idealities introduce compliance and reduce force consistency under high-load contact conditions.

Across all tests, a consistent pattern emerges in which system performance is limited primarily by communication latency rather than computational or control-loop execution speed, with additional practical limitations arising from actuator force capability and transmission behaviour.

Taken together, these results indicate that the system meets its core design objectives, with primary limitations arising from communication latency and actuator force constraints rather than computational performance.

7 Discussion

7.1 Communication and Latency Constraints

The experimental results presented in Section 6.1 demonstrate that system performance is fundamentally constrained by communication latency rather than computational limitations. Although the system achieves high update rates of approximately 1 kHz and low host-side processing latency (sub-millisecond, as shown in Section 6.1.3), the measured round-trip communication delay of approximately 15–16 ms (Section 6.1.2) dominates the closed-loop response.

In haptic rendering systems, achievable virtual stiffness is strongly limited by loop delay. Prior work typically assumes servo update rates on the order of 1 kHz with minimal latency in order to render stiff contact interactions. However, the results presented here show that even when high update rates are maintained, the presence of significant communication delay imposes a practical upper bound on stable stiffness. Given a nominal update period of approximately 1 ms, the measured round-trip delay is more than an order of magnitude larger than the control timestep, introducing substantial phase lag into the feedback loop.

This limitation arises because delayed force feedback introduces phase lag into the closed-loop system, reducing stability margins and increasing the risk of oscillatory behaviour during contact. As a result, the system cannot safely render very high stiffness values without additional stabilisation mechanisms.

Nevertheless, the results demonstrate that stable haptic interaction remains achievable despite this latency, provided that appropriate architectural and control strategies are employed. This reflects a practical distinction between theoretical haptic rendering assumptions and real-world system constraints, where communication delays are often unavoidable.

A likely contributing factor to the observed communication delay is the use of the ST-Link virtual COM interface for host–device communication. This interface was selected due to its simplicity and ease of integration during development, allowing rapid prototyping without requiring additional

hardware or low-level USB implementation. However, the burst-like packet delivery observed in Section 6.1.3, together with the approximately 15–16 ms round-trip latency, suggests that this interface introduces buffering and scheduling overhead that is significant relative to the 1 kHz control timestep.

While the present work does not isolate the exact contribution of each component in the communication stack, it is likely that a substantial portion of the measured latency arises from the virtual COM transport layer rather than host-side computation or control loop execution. Alternative communication approaches, such as direct UART communication via a dedicated USB–UART bridge or lower-latency interfaces, could therefore potentially reduce the round-trip delay and improve achievable haptic stiffness.

These results highlight that, in practical haptic systems, communication architecture can be a dominant performance constraint, and must be considered alongside control and mechanical design when targeting high-fidelity interaction.

7.2 Stability of the Haptic Interaction

Despite the presence of significant communication latency and non-uniform packet arrival behaviour observed in Section 6.1.3, the system was able to maintain stable haptic interaction under a wide range of operating conditions. This stability can be attributed to several key design choices in the system architecture.

Firstly, the use of snapshot-based communication channels ensures that the haptic rendering loop always operates on the most recent available device state, rather than processing queued historical data. This effectively mitigates the impact of burst-like packet delivery observed in the communication results, preventing outdated state information from introducing additional delay or instability into the control loop.

Secondly, the separation of communication, haptic rendering, and physics simulation into independent threads allows each subsystem to operate at an appropriate rate without blocking time-critical operations. This decoupling reduces the propagation of timing variability between subsystems.

In addition, stability was significantly improved through the use of damping and torque rate limiting at the embedded control level, as evidenced by the reduction in oscillatory behaviour observed during hardware validation (Section 6.3.1). The virtual coupling model introduces dissipative be-

haviour that helps to counteract the destabilising effects of delayed feedback, while the slew-rate limiter constrains rapid changes in commanded torque. This prevents abrupt force discontinuities that could otherwise excite oscillations in the system.

Together, these mechanisms enable the system to maintain stable and controllable interaction despite communication delays that would otherwise limit performance in a purely high-stiffness haptic rendering framework.

7.3 Hardware Limitations

The performance and evaluation of the system were subject to several hardware-related limitations arising from practical integration challenges and resource constraints during development. These limitations are directly reflected in the qualitative interaction behaviour observed during hardware validation (Section 6.4), including the presence of compliant contact behaviour and variation in rendered force.

A primary limitation was the absence of current sensing in the actuator control system. The initial design intended to utilise integrated motor driver solutions based on the STM32 B-G431-ESC1 platform, which provide built-in current sensing capabilities suitable for accurate torque control. However, significant integration challenges were encountered when attempting to interface these drivers with the selected magnetic encoders, particularly in achieving reliable and stable operation within the required control framework.

Due to the increased implementation complexity and project time constraints, a design decision was made to adopt SimpleFOC Mini driver modules, which provided a more straightforward and reliable integration pathway with the STM32 Nucleo development board. This allowed development effort to be focused on the embedded control and host-side software architecture, which formed a central objective of the project.

However, the use of SimpleFOC Mini drivers without current sensing introduces an important limitation: commanded torque cannot be interpreted as a calibrated physical torque output. Instead, torque control operates effectively open-loop with respect to current, making the output dependent on motor characteristics, supply conditions, and load variations. As a result, the system provides qualitative rather than quantitatively accurate force feedback.

Despite this limitation, the force feedback was sufficient to convey relative interaction forces and

contact conditions to the user. For the intended application as a training and prototyping interface, this qualitative behaviour was considered an acceptable trade-off.

A second limitation arises from the mechanical transmission used to increase actuator torque. A GT2 timing belt reduction was implemented on the base joint to compensate for the limited torque output of the selected low-cost gimbal motors. While effective in increasing available torque, this transmission introduced non-ideal behaviour under high load conditions.

During interaction with large rendered forces, the belt was observed to slip when the applied load exceeded the transmission capability. An adjustable idler pulley was introduced to increase belt tension and reduce slip; however, this did not fully eliminate the issue. Residual slip is likely due to the use of 3D-printed pulleys, which do not achieve the same tooth profile precision and engagement as manufactured components.

This behaviour limits the maximum renderable force and introduces non-linearities into the transmission, reducing the consistency of force feedback under high load conditions.

In addition, the overall torque capability of the selected motors was found to be insufficient for rendering very stiff virtual environments. Even in the absence of belt slip, the actuators were unable to generate sufficiently large torques to convincingly simulate hard contact conditions. As observed in Section 6.4.2, this results in noticeable compliance during interaction with virtual walls, where force increases gradually rather than instantaneously.

A further limitation arose from the failure of one actuator during development. The exact cause of this failure could not be determined, and the actuator was not replaced within the available project timeframe and budget. Consequently, final hardware validation was conducted using a single actuated joint, while the second joint remained available for position sensing only, as described in Section 6.4.

Although this did not affect the system architecture or communication framework, it reduced the system from full two-degree-of-freedom force feedback to partial actuation during physical testing.

Overall, these results demonstrate that, in addition to communication latency, actuator capability and transmission design impose significant constraints on achievable haptic performance in low-cost systems.

7.4 Design Trade-Offs

The design of the system reflects a set of trade-offs between stability, responsiveness, and achievable force fidelity.

While the use of damping and torque rate limiting was effective in ensuring stable interaction, these mechanisms introduce a reduction in perceived stiffness and responsiveness. In particular, the rate limiting of actuator torque results in a finite rise time when contact forces are applied. This leads to a short delay between initial contact and the development of the full resistive force, which reduces the sharpness of hard contact interactions such as collisions with rigid virtual walls, as observed during hardware validation (Section 6.4.2).

As a result, high-speed interactions and abrupt contacts are not rendered with the same fidelity as would be expected in systems with lower latency and higher achievable stiffness. Instead, contact forces are perceived as gradually increasing rather than instantaneous, which can affect realism in scenarios involving rigid constraints.

However, this trade-off is necessary to ensure stable behaviour in the presence of communication delay. Without rate limiting and damping, the system exhibited oscillatory and unstable behaviour during contact, particularly under high stiffness settings, as demonstrated in Section 6.3.1. The implemented approach therefore prioritises stability and controllability over maximum stiffness, which is appropriate for a low-cost prototype system.

These results highlight a key design consideration in practical haptic systems: achieving stable interaction under real-world constraints often requires compromising on ideal force rendering performance.

8 Conclusions and Future Work

8.1 Conclusions

This project presented the design, implementation, and evaluation of a low-cost, modular haptic rendering system integrating a two-degree-of-freedom input device with a custom real-time simulation and visualisation framework.

The system successfully achieved its primary objective of enabling bidirectional interaction between a user-operated device and a virtual environment, with real-time computation of interaction forces and feedback through actuator torque control. The architecture was designed to prioritise modularity and extensibility, allowing clear separation between hardware, communication, simulation, and rendering subsystems.

Experimental results demonstrated that the system is capable of sustaining high update rates of approximately 1 kHz, with negligible packet loss at higher transmission frequencies and low host-side processing latency. However, communication analysis revealed a consistent round-trip delay of approximately 15–16 ms, which dominates the closed-loop response and imposes a fundamental limitation on achievable haptic performance.

Despite this latency, stable and controllable haptic interaction was achieved through a combination of architectural and control strategies. The use of snapshot-based communication ensured that the system operated on the most recent available data, mitigating the effects of burst-like packet delivery. In addition, the application of damping and torque rate limiting was critical in maintaining stability, preventing oscillatory behaviour during contact, and enabling sustained interaction with virtual constraints.

Simulation and hardware validation confirmed that the system is capable of rendering physically consistent contact and constraint forces. The virtual coupling and proxy-based rendering approach produced expected unilateral contact behaviour and supported constrained motion along virtual surfaces. Hardware testing further demonstrated that stable interaction could be maintained under real-time conditions, even with communication variability and partial actuation.

However, the results also highlight an important trade-off between stability and force fidelity. While the system remains stable under delayed feedback, the achievable virtual stiffness is limited, and contact forces exhibit a finite rise time rather than instantaneous response. This reduces the realism of high-speed or high-stiffness interactions, particularly for rigid contact scenarios.

Overall, the project demonstrates that stable and usable haptic interaction can be achieved using low-cost hardware and a modular software architecture, even in the presence of significant communication latency. The findings emphasise that in practical systems, communication delay is often the dominant constraint, and that appropriate architectural and control design is essential to achieving stable performance.

8.2 Future Work

Several avenues for future development have been identified to improve system performance, fidelity, and extensibility.

A key priority is the reduction of communication latency. The current system relies on the ST-Link virtual COM interface, which was selected for its simplicity and ease of integration during early development. While this approach enabled rapid prototyping, the results presented in Section 6.1 indicate that the communication pathway introduces buffering and scheduling delays that dominate the closed-loop response.

Future work should therefore investigate lower-latency communication architectures between the host and embedded controller. In particular, direct UART communication via a dedicated USB–UART bridge may reduce buffering effects associated with the virtual COM interface, while alternative approaches such as native USB implementations (e.g., bulk transfer), CAN bus, or Ethernet-based communication could provide more deterministic timing behaviour.

A systematic comparison of these communication approaches would allow the contribution of each component in the communication stack to be isolated and could provide a pathway toward significantly reducing round-trip latency and improving achievable haptic stiffness.

Improving actuator performance and force fidelity is another critical area. The incorporation of current sensing would enable accurate torque control and allow quantitative validation of rendered forces. This would also support the implementation of more advanced control strategies, including impedance control and operational-space control formulations.

Mechanical transmission design could also be improved. While GT2 timing belts provided a simple and low-cost solution for torque amplification, they introduced slip and compliance under high load conditions. Future designs could explore alternative transmission mechanisms such as capstan drives, as demonstrated in low-cost haptic systems such as the Hapkit platform. These approaches offer reduced backlash and improved force transmission characteristics, which could enhance both stability and realism.

The use of higher torque actuators or geared motor systems would further improve the ability to render stiff virtual environments, reducing compliance during contact and enabling more realistic hard-contact interactions.

Restoration of full two-degree-of-freedom actuation is also an important step. Enabling force feedback across both joints would improve interaction realism and allow more complex manipulation and constraint-based tasks to be explored.

Further improvements to the haptic rendering model could address limitations observed under large penetration conditions. More advanced contact handling approaches, such as continuous collision detection, constraint-based solvers, or improved normal estimation techniques, could enhance robustness and force consistency.

From a software perspective, the modular architecture provides a strong foundation for extension toward more advanced simulation environments. Integration with external platforms such as NVIDIA Isaac Sim or MATLAB/Simulink would enable more realistic robotic scenarios and facilitate comparison with established robotic control frameworks.

Finally, a more comprehensive evaluation framework incorporating quantitative force measurement, user studies, and task-based performance metrics would provide deeper insight into the effectiveness of the system as a training tool.

Together, these developments would enable the system to evolve from a low-cost prototype into a more capable and general-purpose haptic interface platform for robotic interaction and training applications.

Page 49 of 49

References

[1] G. S. Giri, Y. Maddahi, and K. Zareinia, “An Application-Based Review of Haptics Technology,” *Robotics*, vol. 10, no. 1, p. 29, Feb. 5, 2021, ISSN: 2218-6581. DOI: 10.3390/robotics10010029 Accessed: Apr. 16, 2026. [Online]. Available: https://www.mdpi.com/2218-6581/10/1/29 (cited on pp. 1, 4, 6).

[2] M. Bergholz, M. Ferle, and B. M. Weber, “The benefits of haptic feedback in robot assisted surgery and their moderators: A meta-analysis,” *Scientific Reports*, vol. 13, no. 1, p. 19 215, Nov. 6, 2023, ISSN: 2045-2322. DOI: 10.1038/s41598-023-46641-8 Accessed: Apr. 16, 2026. [Online]. Available: https://www.nature.com/articles/s41598-023-46641-8 (cited on pp. 1, 5).

[3] D. C. Ruspini, K. Kolarov, and O. Khatib, “The haptic display of complex graphical environments,” in *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques - SIGGRAPH ’97*, Not Known: ACM Press, 1997, pp. 345–352, ISBN: 978-0-89791-896-1. DOI: 10.1145/258734.258878 Accessed: Apr. 16, 2026. [Online]. Available: http://portal.acm.org/citation.cfm?doid=258734.258878 (cited on pp. 1, 6).

[4] R. Adams and B. Hannaford, “Stable haptic interaction with virtual environments,” *IEEE Transactions on Robotics and Automation*, vol. 15, no. 3, pp. 465–474, Jun. 1999, ISSN: 1042296X. DOI: 10.1109/70.768179 Accessed: Apr. 16, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/768179/ (cited on pp. 1, 6).

[5] M. O. Martinez et al., “3-D printed haptic devices for educational applications,” in *2016 IEEE Haptics Symposium (HAPTICS)*, Apr. 2016, pp. 126–133. DOI: 10.1109/HAPTICS.2016.7463166 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/7463166 (cited on pp. 2, 6).

[6] M. O. Martinez et al., “Open source, modular, customizable, 3-D printed kinesthetic haptic devices,” in *2017 IEEE World Haptics Conference (WHC)*, Jun. 2017, pp. 142–147. DOI: 10.1109/WHC.2017.7989891 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7989891 (cited on pp. 2, 7).

[7] “Inverse3 X,” Accessed: Apr. 16, 2026. [Online]. Available: https://www.haply.co/inverse3x (cited on p. 5).

[8] “Geomagic touch haptic device,” Accessed: Apr. 16, 2026. [Online]. Available: https://shop.gomeasure3d.com/products/geomagic-touch-haptic-device (cited on p. 5).

[9] "Geomagic touch X haptic device," Accessed: Apr. 16, 2026. [Online]. Available: https://shop.gomeasure3d.com/products/geomagic-touch-x-haptic-device (cited on p. 5).

[10] "Sales inquiries," Accessed: Apr. 16, 2026. [Online]. Available: https://www.forcedimension.com/contact/sales (cited on p. 5).

[11] C. Zilles and J. Salisbury, "A constraint-based god-object method for haptic display," in *Proceedings 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots*, vol. 3, Pittsburgh, PA, USA: IEEE Comput. Soc. Press, 1995, pp. 146–151, ISBN: 978-0-8186-7108-1. DOI: 10.1109/IROS.1995.525876 Accessed: Apr. 16, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/525876/ (cited on p. 6).

[12] O. Khatib, "A unified approach for motion and force control of robot manipulators: The operational space formulation," *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, Feb. 1987, ISSN: 2374-8710. DOI: 10.1109/JRA.1987.1087068 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/1087068 (cited on pp. 6, 11).

[13] A. Lavatelli, M. Bordegoni, and F. Ferrise, "Design of an open-source low cost 2DOF haptic device," Sep. 2014. DOI: 10.1109/MESA.2014.6935574 (cited on p. 7).

[14] A. Skuric et al., "SimpleFOC: A field oriented control (FOC) library for controlling brushless direct current (BLDC) and stepper motors," *Journal of Open Source Software*, vol. 7, no. 74, p. 4232, 2022. DOI: 10.21105/joss.04232 [Online]. Available: https://doi.org/10.21105/joss.04232 (cited on p. 14).

[15] P. Vas, *Sensorless Vector and Direct Torque Control*. Oxford University Press, May 14, 1998, ISBN: 978-0-19-856465-2. DOI: 10.1093/oso/9780198564652.001.0001 Accessed: Apr. 16, 2026. [Online]. Available: https://doi.org/10.1093/oso/9780198564652.001.0001 (cited on p. 14).

Appendices

A Project outline

Project outline as submitted at the start of the project is a required appendix. Put here.

B Risk assessment

Risk assessment is a required appendix. Put here.

C Code